

Lab 1: Requirements Engineering & UML Use-Case Modelling

Problem Statement #44: Database Query Performance Profiler

Problem Context & Overview

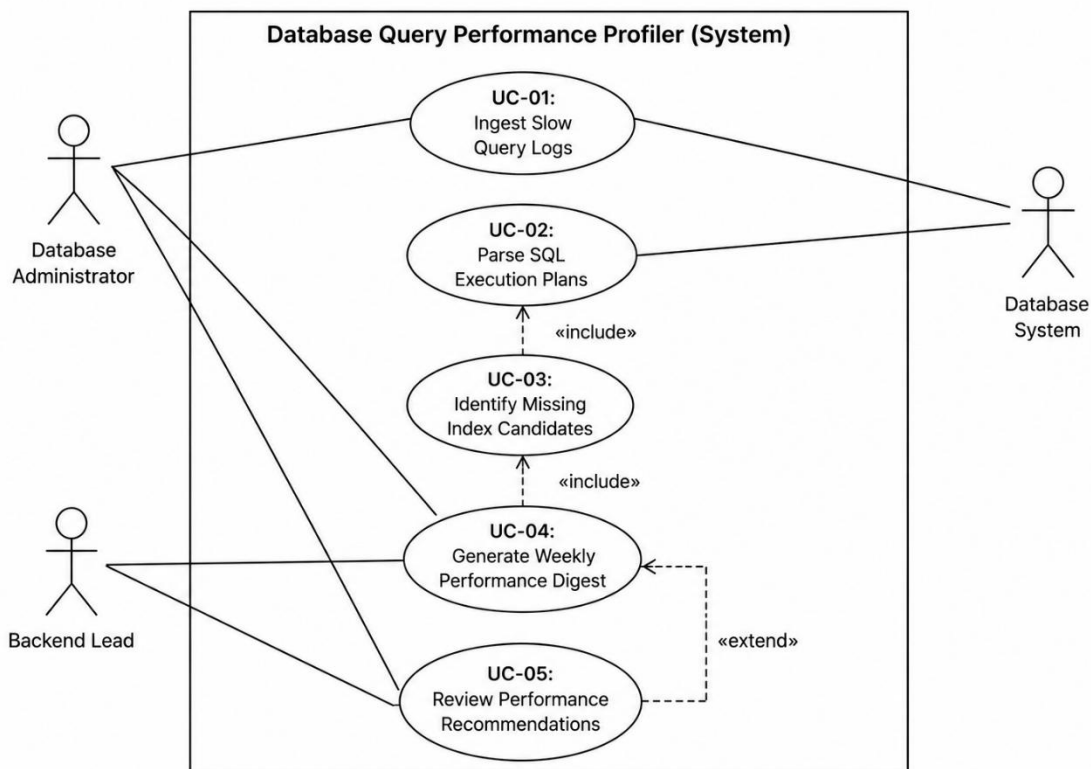
A database observability tool that ingests slow query logs, parses SQL execution plans, identifies missing index candidates, and generates weekly performance optimization digests.

FUNCTIONAL REQUIREMENTS

	TYPE	DESCRIPTION	PRIORITY	Acceptance Criteria	RATIONALIZE
FR-001	FUNCTIONAL	The system shall parse Postgres/MySQL EXPLAIN plans, highlight sequential table scans on large tables, and recommend optimal index column definitions.	HIGH	Pass: Missing composite index is identified on filtered columns. Fail: Query plan with sequential scan is marked as optimized.	Helps identify inefficient queries and recommend suitable indexes
FR-002	Functional	The system shall ingest and parse slow query logs from the database.	High	Pass: Valid slow query log records are successfully ingested and parsed. Fail: Valid records are rejected or lost.	Required to analyze slow database queries.
FR-003	Functional	The system shall identify missing index candidates from SQL queries and execution plans.	High	Pass: The system identifies an appropriate missing index candidate for a query. Fail: No candidate is identified when a suitable index is missing.	Helps improve query performance through indexing.

FR-004	Functional	The system shall generate a weekly performance optimization digest containing identified performance issues and recommendations.	Medium	Pass: A weekly digest containing performance issues and recommendations is generated. Fail: The digest is missing required optimization information.	Provides regular performance insights to stakeholders.
FR-005	Functional	The system shall allow the Database Administrator to review query performance findings and optimization recommendations.	Medium	Pass: The Database Administrator can view the identified issues and recommendations. Fail: Findings or recommendations cannot be accessed.	Helps administrators review and act on performance problems.
NFR-001	Nonfunctional	The system shall handle up to 5,000 slow query log records per minute without dropping entries or slowing the host database	High	Pass: Benchmarking confirms processing of 5,000 records per minute without dropped entries or unacceptable host DB impact. Fail: Records are dropped or the host database is significantly slowed.	Ensures reliable performance under peak load.

NFR-002	Nonfunctional	The system shall protect database performance data from unauthorized access.	High	Pass: Unauthorized users are denied access to performance data. Fail: An unauthorized user can access the data.	Protects sensitive database performance information.
---------	---------------	--	------	--	--



Use-Case Flow Specification

Use Case: UC-01 – Ingest Slow Query Logs

Preconditions

1. The Database Query Performance Profiler is running.
2. Slow query logs are available from the database.
3. The system has access to the configured database log source.

Postconditions

1. Slow query log records are successfully ingested.
2. The ingested records are available for further analysis.
3. Invalid or corrupted records are reported without stopping the ingestion process.

Main Success Scenario

1. The Database Administrator initiates slow query log ingestion.
2. The system connects to the configured database log source.
3. The system reads the available slow query log records.
4. The system validates the log records.
5. The system parses the valid log records.
6. The system stores the parsed records for performance analysis.
7. The system confirms successful ingestion.
8. The use case ends successfully.

Alternate Flow – Invalid Log Record

1. During validation, the system detects an invalid or corrupted log record.
2. The system rejects the invalid record.
3. The system records an error message for the rejected record.
4. The system continues processing the remaining valid records.
5. The system reports the number of rejected records after ingestion is completed.